Comparative Study of ECG Classification Methods

July 10, 2026

Contents

Student:
FAYADAS Hugo

Program:
LDD3 MPSI

Supervisor:
Matthieu Kowalski

Abstract

The purpose of this paper is to display and compare classification processes applied to electrocardiogram (ECG) signals. With a main focus on the convolutional dictionary learning (CDL) process. We propose to apply a list of classical classifiers on the results of this process. We compare the estimation errors made with models trained on raw signals, reconstructed signals through CDL and atom activation times. Also we evaluate the performance of models based on features of the signal, wavelet transform and interest ourselves in the influence of data conditioning on the results. We provide a complete and executable Python code for the reader's benefit (see Appendix).

Introduction

Electrocardiography is a medical examination aimed at measuring the electrical activity of the heart, resulting in a tracing, the electrocardiogram (we will use the two terms interchangeably hereafter). It is mainly used for the detection and diagnosis of numerous pathologies such as myocardial infarction, arrhythmia, fibrillation...

However, detecting abnormalities within an electrocardiogram (ECG) is complex, requiring time and qualified personnel, making these essential diagnostics difficult to access. Numerous studies have been conducted to automate this process. The objective of this study is to compare autonomous methods for determining pathologies. We will focus mainly on a method based on Convolutional Dictionary Learning (CDL). We will then compare the performance of different classifiers applied to the results obtained by CDL, as well as to raw ECGs, a set of their features, their local maxima, and their wavelet transforms. We are also interested in the influence of data conditioning on classification performance.

1 Electrocardiogram Signals

This section is dedicated to defining the object of this study: electrocardiogram signals. An electrocardiogram is the cutaneous recording of the electrical activity of the heart. In practice, it is performed by placing twelve electrodes on the skin, obtaining 12 leads that allow the signal to be reconstructed spatially. In this study, we will confine ourselves to a single lead, i.e., a single-channel ECG. Heart contractions are triggered by an electrical depolarization wave passing through the different cavities. A heartbeat can then be decomposed into a sequence of conventional waves, as can be seen in Figure 9.

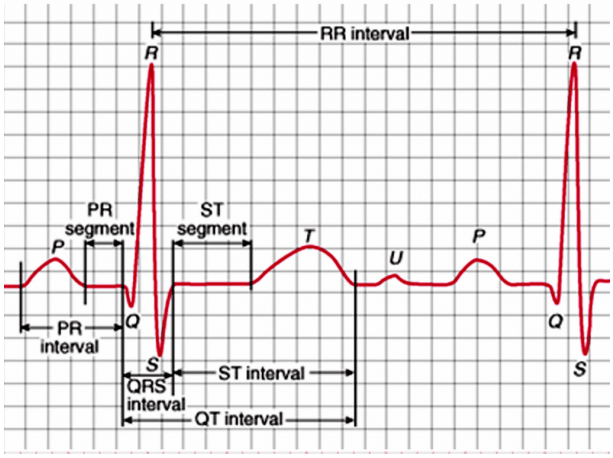


Figure 1: Diagram of a typical heartbeat.

We distinguish the P wave, which represents the contraction or depolarization of the atria; the QRS complex, the contraction of both ventricules, which is the sharpest and highest peak

and the most easily detectable element. Finally, the T wave is the relaxation or repolarization of the ventricles in preparation for a new heartbeat, which is sometimes followed by the smaller and shorter U wave, whose origin is still debated, see [4].

For this study, we have a single-channel ECG database containing 162 ECGs sorted by professionals, freely accessible on Physionet and downloadable here. Each of them is sampled at a frequency of 128Hz and has a duration of 512s. We have three types of signals: NSR type signals for "Normal Sinus Rhythm" (i.e., a healthy ECG), ARR type signals for "Arrhythmia", and CHF for "Congestive Heart Failure". The three types of signals are shown in Figure 2. We have 96 ARR-type signals, 36 NSR-type signals, and 30 CHF-type signals.

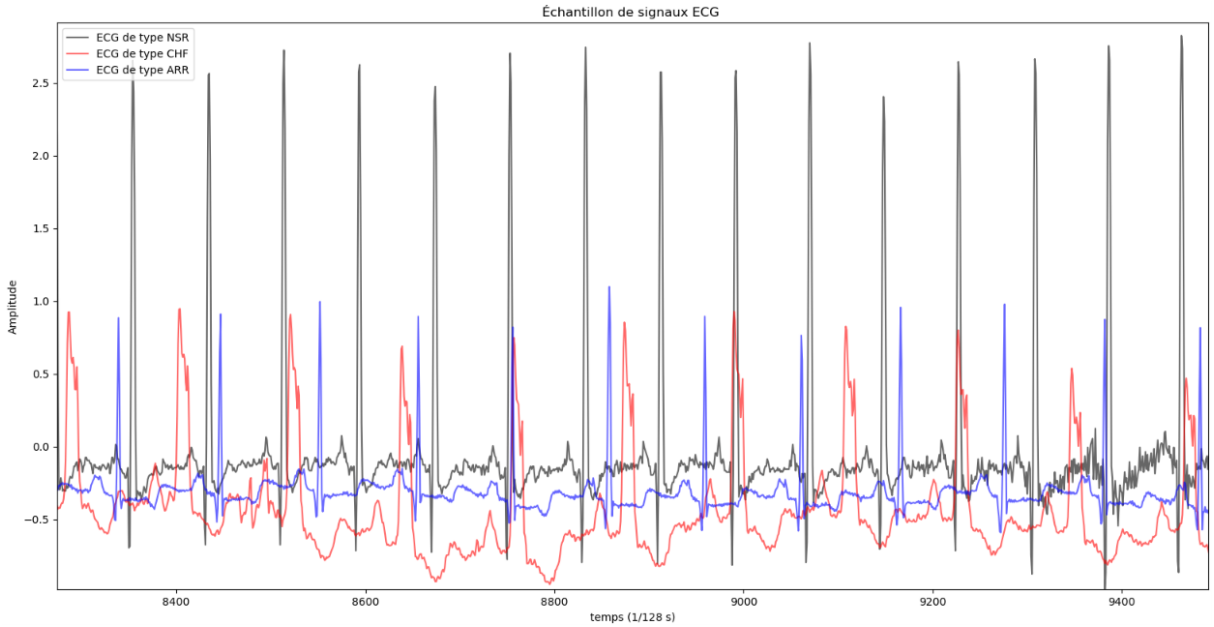


Figure 2: Extract of ECG signals of each type.

In this figure, we observe marked differences between the signals, but above all, a similar pattern seems to repeat for each of them.

2 Convolutional Dictionary Learning

In this section, we focus on the Convolutional Dictionary Learning (CDL) algorithm. First, we will define the concept of CDL before utilizing it on the ECG data presented previously.

2.1 Theory

The objective of Convolutional Dictionary Learning is to find a sparse representation of the data—that is, a representation consisting primarily of zeros, allowing for signal compression and highlighting the most significant features. To achieve this, we aim to represent each signal as combinations of basic elements shared by all signals of the same category. These elements are called atoms and form a dictionary. The goal is to simultaneously minimize the discrepancy between the data and its representation while making it as sparse as possible. These concepts can be formalized as follows: let X be the signals of a certain type where $X \in \mathbb{R}^{n\,trials \times n\,times}$, where n_trials and n_times represent the number of signals and their length respectively. Let D be the dictionary whose columns are the atoms, which is the matrix of combination coefficients used to reconstruct X. Here $D \in \mathbb{R}^{n\,atoms \times n\,times\,atoms}$, where n_atoms is the number of atoms, and n_times_atoms is their length, which are parameters to be set. Finally, Z is the sparse representation tensor, with $Z \in \mathbb{R}^{n\,trials \times n\,atoms \times times\,atoms}$. The optimization problem is then

written as:

$$\min_{D,Z} \frac{1}{2} \sum_{i=1}^{n\ trials} \left\| X_i - \sum_{k=1}^{n\ atoms} d_k * z_{i,k} \right\|_2^2 + \lambda \sum_{i=1}^{n\ trials} \sum_{k=1}^{n\ atoms} \|z_{i,k}\|_1$$
$$\text{subject to: } \|d_k\|_2^2 \leq 1 \quad \forall k \in \{1, \ldots, n\ atoms\}$$

The first term of this formula aims to ensure the correspondence between the representation and the data. Convolution allows atoms to be replicated along the signal, while the $z_{i,k}$ specific to each signal indicate when and with what amplitudes to replicate them; these are "activation maps" or activation signals. The second term serves to enforce sparseness on these maps (also called activation times). The lambda parameter controls the weight given to this sparsity: for a very high lambda, the model will attempt to put zeros everywhere, even if it results in just a straight line; conversely, a lambda that is too small would lead to atoms activating continuously, losing the advantages of this representation. A schematic example of this process is shown in Figure 3.

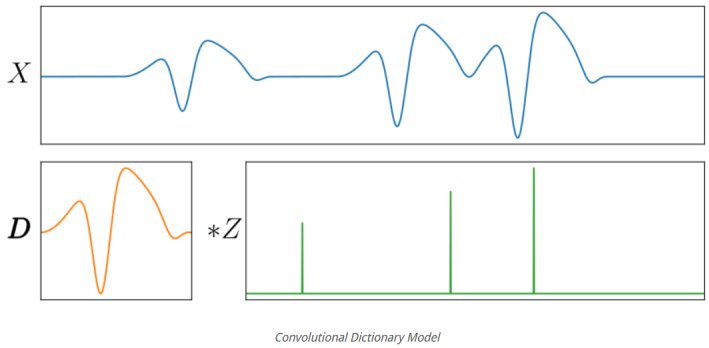


Figure 3: Decomposition of a noise-free one-dimensional signal (blue) as a convolution between a temporal pattern/atom (orange) and a sparse activation signal (green).

To optimize this problem, we alternately fix D and Z and estimate the other using gradient descent for the quadratic term and a more complex algorithm for the second, involving a thresholding function[1]. For more details, see [8] and [3]. To reconstruct a signal, it is then sufficient to perform the convolution product between the atoms of its category and its own activation times. That is,

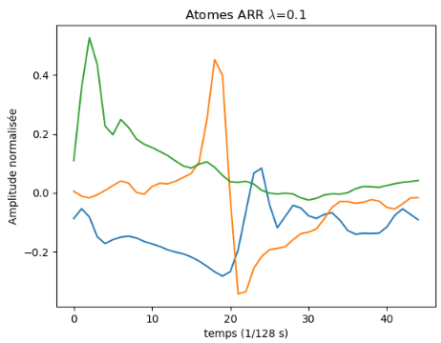
$$X_i = \sum_{k=1}^{n\ atoms} z_{i,k} * d_k$$

2.2 Application to ECG Signals

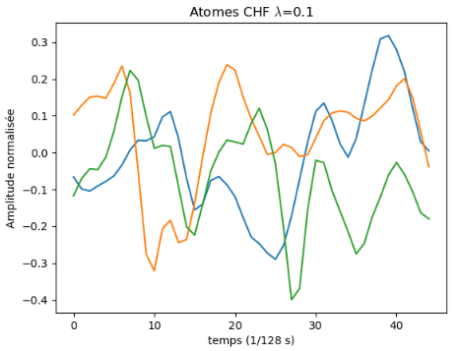
In practice, we use the Python module alphacsc[8],[2], which contains the "learn_d_z" function that solves the previous optimization problem based on parameters to be set. These parameters are: the number of algorithm iterations, the number of atoms to search for, their length, and the sparsity coefficient λ. To set these parameters, we draw inspiration from the raw signals and the patterns detected in them (see Figure 2) and proceed by trial and error until a coherent result is achieved, especially for signal reconstruction.

We present here the best results obtained, corresponding to 50 iterations and 3 atoms of length 45 measurements (i.e., 45/128=0.35s). We distinguish two sparsity coefficients λ to show the impact of sparsity on the results. Figure 4 presents the atoms obtained when $\lambda = 0.1$, while Figure 6 shows $\lambda = 1$. We can already see differences in the shape of the atoms, which are much more detailed in the first case, made possible by less sparse activation signals. Figures 5 and 7 show samples of reconstructed signals of all types with $\lambda = 0.1$ and $\lambda = 1$ respectively. We can once again observe that in the first case, the approximation is much closer to the original data.

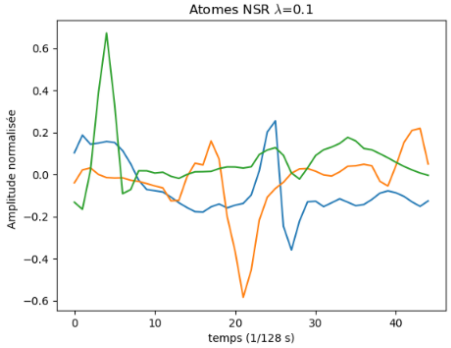
[1]Since this algorithm is not the subject of this study and the corresponding functions are already available, we will not dwell on it further



(a) Atoms for arr type signals

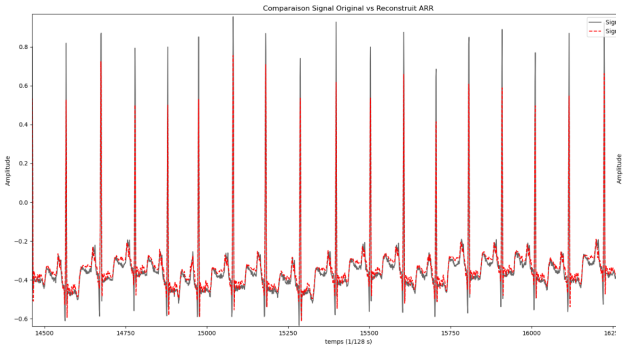


(b) Atoms for chf type signals

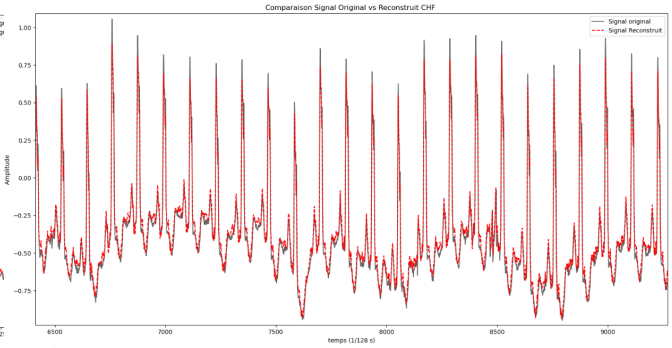


(c) Atoms for nsr type signals

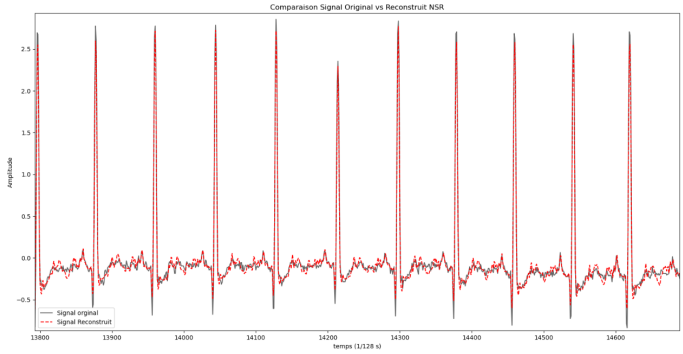
Figure 4: Atoms obtained for $\lambda = 0.1$



(a) Comparison of arr type signals



(b) Comparison of chf type signals

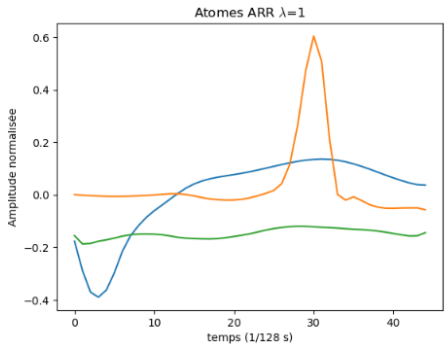


(c) Comparison of nsr type signals

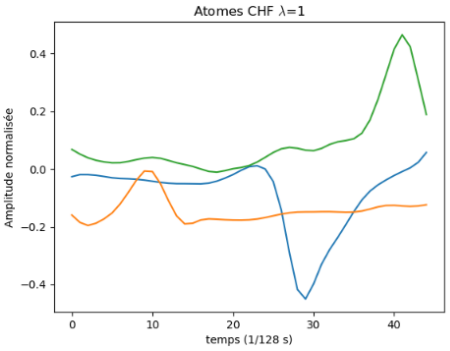
Figure 5: Comparison of raw and reconstructed signals with $\lambda = 0.1$

3 Classifications

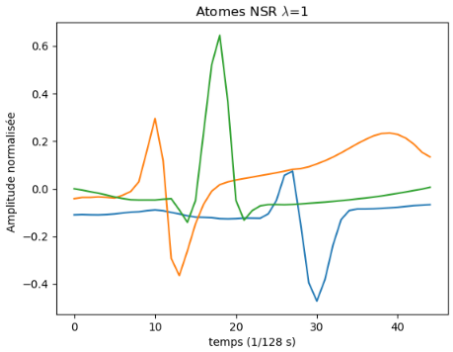
The objective of this section is to compare the performance of classifiers in determining the types of ECG signals. For this purpose, we provide a brief definition of the classifiers here.



(a) Atoms for arr type signals

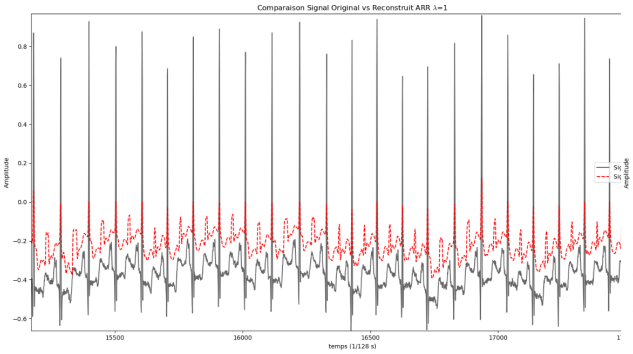


(b) Atoms for chf type signals

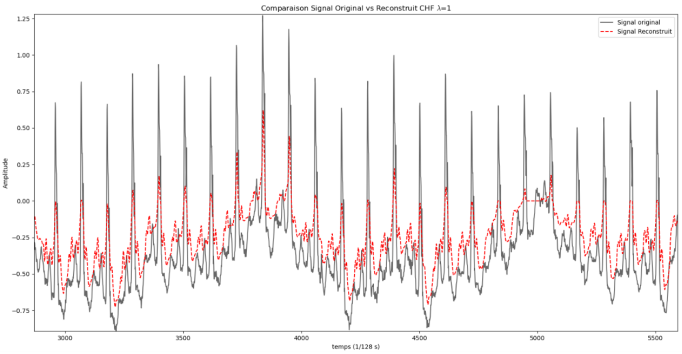


(c) Atoms for nsr type signals

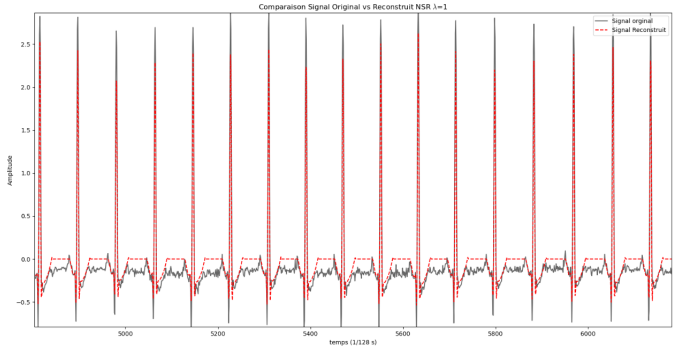
Figure 6: Atoms obtained for $\lambda = 1$



(a) Comparison of arr type signals



(b) Comparison of chf type signals



(c) Comparison of nsr type signals

Figure 7: Comparison of raw and reconstructed signals with $\lambda = 1$

We outline the procedures for evaluating their performance before applying them to the data established in the previous section. We then define methods to form new features from the raw signals. First, we determine the times corresponding to the signal peaks, and then we extract a set of characteristics (features). We apply the classifiers to these new elements and compare the

results acquired. Afterward, we use the wavelet transform and autoregressive models to obtain a new dataset on which we apply the classifiers. Finally, we present a summary and compare the performance obtained with the best classifier.

3.1 Definition of Classifiers

By themselves, the calculations performed previously do not allow for the automated determination of a signal's type. For this, we turn to classifiers. A classifier is a prediction algorithm that has been trained by presenting it with a large amount of data along with the corresponding labels; when presented with new, unknown data, it can predict the corresponding label using what it has learned. This makes classifiers specifically interesting for our study. The most commonly used classifiers are those from the Python scikit-learn module[5],[1], which are the ones we will use hereafter. The classifiers we will use are the following: LogisticRegression, DecisionTreeClassifier, RandomForest, GaussianNB, BernoulliNB, KNeighbors. It should be noted that some are initially defined as binary classifiers (only able to separate 2 categories); however, we decide to retain them for the rest of the study to observe the impact this has on their performance, knowing that they naturally tend to proceed by opposing all categories against one successively for each category.
Given the respective performance of the classifiers during the tests, we briefly present here the theoretical operation of one of the classifiers that proved to be the most performant. The RandomForest classifier is based on the principle of decision trees: we move through the tree by answering basic questions at each level. For example, if we wanted to make a decision tree for means of transportation, one of the first questions could be "does the vehicle have wheels?"; if yes, a second question: "more than two wheels?"; if no, it is probably a motorcycle, and so on. However, decision trees are often faced with the problem of overfitting; they learn the training data perfectly but are too complex and detailed to be useful on new data. To address this, instead of growing a single tree, RandomForest grows a large number of average trees that are not trained in exactly the same way. They only have access to a random subset of the data, and moreover, they can only base their decisions on a random subset of features at each split rather than the whole set. Thus, not all trees give the same answer, and when classifying new data, the label that receives the most votes among the trees wins. This operating mode makes RandomForest one of the most robust classifiers in existence.

3.2 Evaluation of Classifiers

To evaluate the performance of the classifiers in the following tests, we will compute scores for each of them. These scores are the error rate, precision, recall, and F1-score. The precision of a model for a category is defined as the ratio of the number of times it was correctly predicted for the test dataset to the total number of times it was predicted on that same set. Recall corresponds to the ratio between the number of times a class was correctly predicted within the test set and the number of times this class actually appears in the test set. To better understand the difference between the two, imagine that our model always returns the same class; then the recall for this class would be 100%, as all data belonging to this class would have been found. The precision, however, would be very poor since we would have associated a large amount of unrelated data with this class. Ideally, therefore, we want to maximize both scores simultaneously, which is why we calculate the F1-score. It is the harmonic mean of the previous scores. Finally, the error rate (also denoted as err rate) is the number of misallocated labels divided by the size of the test set; it contains less detail than the others but allows for quick comparisons between classifiers.

3.3 First Applications

For comparison purposes, we will apply the classifiers to raw signals, their activation times, and reconstructed signals. To apply classifiers to our data, we need to split it into training and test data. We randomly select a high percentage, typically between 70% and 90%, of

each of the three classes of studied signals, shuffle them randomly, and obtain the training set. We obviously retain the corresponding label for each signal. The remainder of the signals are randomly allocated as the test set. We then apply the classifier to the data thus formed and record the scores mentioned previously. We average these scores by repeating this entire process one hundred times per item. We decide to see if pairing two categories and opposing them to the third yields better results. For this, we define a sorting variable that takes the values: classic, chf, nsr1, and nsr2. They respectively designate the case where we separate the three variables, the case where we combine the ARR and CHF signals together (under the CHF label), the case where we combine the ARR and NSR signals together (under the NSR label), and finally the case where we combine the NSR and CHF signals together (under the NSR label). In the remainder of this study, we only focus on results for which $\lambda = 0.1$, as the results with $\lambda = 1$ were systematically worse. Tables 1, 2, and 3 present the rates corresponding to each model and configuration. First, we notice that the results from raw signals are very poor, as expected, because they contain too many data points for the classifiers to be effective. We will therefore no longer focus on this configuration in the remainder of this study. Secondly, we find that the RandomForest classifier is often better. Furthermore, we note that none of the classifiers for the 2 datasets give satisfactory results for the 'classic', 'chf', and 'nsr2' sorting modes. Conversely, when based on activation times, we obtain very good results for the 'nsr1' sorting mode. This suggests that ARR and NSR type signals have very similar activation times, meaning they do not hold the majority of the information. On the contrary, the activation times of CHF signals seem to be quite different and carry a certain amount of information. We can confirm this impression by analyzing the other scores. In Figure 8, we compare the scores obtained for the LogisticClassifier with activation times for the classic and nsr1 sorting modes. For the classic sorting mode, we can see low scores for NSR signals[2]. This confirms the idea that activation times contain little information for this type of signal. On the other hand, when switching to the nsr1 sorting mode, we obtain much better scores[3], a trend that is confirmed in the other score tables (see Appendix).

Classifier	**Error Rate (Classic Sorting)**
Logistic Regression	0.444
Decision Tree	0.437
Random Forest	0.281
Gaussian NB	0.627
Bernoulli NB	0.488
KNN ($k = 10$)	0.379
KNN ($k = 11$)	0.381
KNN ($k = 12$)	0.378

Table 1: Comparison of error rates with raw signals and classic sorting

(a) Scores for LogisticClassifier with classic sorting

	Precision	**Recall**	**F1_Score**
A	71.689944	97.284211	82.489666
C	91.865079	86.695238	88.343717
N	66.833333	13.053571	21.271212

(b) Scores for LogisticClassifier with nsr1 sorting

	Precision	**Recall**	**F1_Score**
C	93.926587	82.704762	86.864205
N	96.352148	98.628612	97.434692

Figure 8: Extract of score tables for activation times $\lambda = 0.1$

[2]In these tables, signal types are indicated only by their first letter

[3]Recall that within the nsr1 sorting mode, ARR and NSR signals are grouped under the NSR label

Model (Activation)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.245	0.215	0.043	0.236
Decision Tree	0.344	0.181	0.163	0.312
Random Forest	0.173	0.128	0.079	0.245
Gaussian NB	0.411	0.238	0.175	0.290
Bernoulli NB	0.261	0.204	0.159	0.315
KNN ($k = 10$)	0.338	0.236	0.078	0.329
KNN ($k = 11$)	0.331	0.234	0.090	0.328
KNN ($k = 12$)	0.346	0.233	0.087	0.350

Table 2: Comparison of error rates with activation times.

Model (Reconstruct)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.407	0.234	0.164	0.445
Decision Tree	0.338	0.185	0.235	0.328
Random Forest	0.281	0.157	0.167	0.280
Gaussian NB	0.404	0.261	0.210	0.417
Bernoulli NB	0.559	0.332	0.434	0.556
KNN ($k = 10$)	0.262	0.165	0.162	0.261
KNN ($k = 11$)	0.285	0.141	0.172	0.258
KNN ($k = 12$)	0.296	0.194	0.159	0.273

Table 3: Comparison of error rates on reconstructed signals.

3.4 Peak Determination and Feature Extraction

We decide to use the raw data to calculate new features. First, we focus on the QRS peaks of each heartbeat; we define a threshold function and extract the coordinates corresponding to the signal peaks. We apply the entire classification process described above to the new data obtained.

Second, we decide to extract fundamental features from each of the signals. The extracted features are: energy, variance, maximum and minimum values, zero-crossing rate, quantiles, frequencies, mean frequency, evolution dynamics via the derivative, skewness, and kurtosis. Skewness and kurtosis, also known as coefficients of asymmetry and kurtosis, measure respectively the symmetry of the signal relative to its mean and the distribution of points in the tail of the curve relative to a normal distribution. Once again, we apply the full classification process to these.

Model (Peaks)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.447	0.254	0.249	0.367
Decision Tree	0.437	0.228	0.257	0.398
Random Forest	0.379	0.195	0.217	0.336
Gaussian NB	0.627	0.583	0.170	0.406
Bernoulli NB	0.488	0.424	0.189	0.371
KNN ($k = 10$)	0.379	0.217	0.199	0.325
KNN ($k = 11$)	0.381	0.218	0.190	0.322
KNN ($k = 12$)	0.378	0.225	0.195	0.335

Table 4: Comparison of error rates with peak analysis.

Model (Features)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.292	0.098	0.211	0.264
Decision Tree	0.162	0.045	0.132	0.158
Random Forest	0.159	0.045	0.118	0.144
Gaussian NB	0.555	0.380	0.217	0.456
Bernoulli NB	0.418	0.235	0.185	0.415
KNN ($k = 10$)	0.416	0.233	0.203	0.401
KNN ($k = 11$)	0.403	0.235	0.207	0.392
KNN ($k = 12$)	0.402	0.229	0.207	0.385

Table 5: Comparison of error rates with extracted features.

3.5 Performance Comparison

From Table 4, we can already notice that no classifier and no sorting configuration yields good results from the QRS peaks, making them poor indicators. Conversely, overall, models based on features yield better results (see Table 5), especially the first three classifiers in the chf sorting configuration. This would imply that information for NSR type signals is mainly contained within at least some of the explored features. Furthermore, by cross-referencing the information provided by these models on features in chf sorting with the results obtained by activation times for nsr1 sorting, we can determine the three categories with good precision.

Input Type	Classic	CHF	NSR1	NSR2
Raw	0.281	-	-	-
Activation Time	0.173	0.128	0.079	0.245
Peaks	0.379	0.195	0.217	0.336
Features	0.159	0.045	0.118	0.144
Reconstruct	0.281	0.157	0.167	0.280

Table 6: Evolution of Random Forest error rate according to input type and sorting method.

Signal Type	Classic	CHF	NSR1	NSR2
Activation Time	0.319	0.211	0.109	0.301
Peaks	0.440	0.294	0.210	0.358
Features	0.326	0.188	0.186	0.323
Reconstruct	0.366	0.208	0.200	0.363

Table 7: Average error rate of all classifiers by input type and sorting method.

Table 7 presents the average error rates of all classifiers by input and sorting type. Clearly, models based on activation times and features are the most performant. Activation times allow for better predictions for the NSR1 sort, while features yield the best results for the CHF sort. This confirms our previous hypotheses regarding information distribution. To determine with certainty the most performant classifier, Table 8 represents the average error rate of each algorithm by sorting type, averaged over all input forms. It then becomes clear that Random Forest is the best of all these classifiers for our problem. We therefore dedicate Table 6 to presenting the evolution of the Random Forest error rate according to the input type and sorting method. We naturally find excellent performance for features in CHF and activation times in NSR1. Furthermore, Table 8 shows that the Gaussian NB and Bernoulli NB models, i.e., the Naive Bayes patterns, are generally quite poor and struggle to assimilate the complexity of the data; their probabilistic approach is likely unsuited to our data. This is particularly true regarding the assumptions made by these models. The independence assumption (the naivety)

Classifier	Classic	CHF	NSR1	NSR2
Logistic Regression	0.348	0.200	0.167	0.328
Decision Tree	0.320	0.160	0.197	0.299
Random Forest	0.248	0.131	0.145	0.251
Gaussian NB	0.499	0.366	0.193	0.392
Bernoulli NB	0.432	0.299	0.242	0.414
KNN ($k = 10$)	0.349	0.213	0.161	0.328
KNN ($k = 11$)	0.350	0.207	0.165	0.325
KNN ($k = 12$)	0.356	0.220	0.162	0.336

Table 8: Average error rate of each algorithm by sorting type, averaged over all input forms.

supposes that signal characteristics are independent of each other, which is almost always false in the inputs we use. On the other hand, these classifiers respectively make Gaussian and binomial assumptions about the data, which is also false. Finally, if we look at KNeighbors or k-nearest neighbors, we see that the choice of k (number of neighbors to consider) does not significantly influence the results.

Signal Representation	Average Execution Time
Raw Signals (*Raw*)	$\sim$ 466.5 seconds
Activation Time	$\sim$ 914.9 seconds
Peaks	$\sim$ 632.8 seconds
Features	$\sim$ 42.1 seconds
Reconstruct	$\sim$ 385.4 seconds

Table 9: Impact of input type on the average training and evaluation time per model (Classic Sort).

Another important element to consider is the computation time required to obtain each of these results. Table 9 indicates the computation time[4] averaged over the classifier models by input type for the classic sort of the classification process as a whole (the 100 iterations). We conclude that features offer the best cost/performance ratio. Although activation times are generally more precise, they are also the most expensive in terms of execution time, nearly 22x the average execution time of features. It should also be remembered that determining atoms and activation times is already a rather time-consuming process. Table 10 focuses on the computation times of the Random Forest classifier, where we observe that activation times require 12 minutes of computation, while features take only 5 seconds. However, this finding must be put into perspective: what takes the most time here is training the models and the fact that the entire process is iterated 100 times. In practice, the model is trained only once and then used to predict labels one by one; it predicts the class of a patient's ECG.

3.6 Wavelet Transform and Special Indicators

When taking signal features, we calculate, among other things, their Fourier transform. Although the transform is useful, it has a major drawback: it provides the frequencies that appear in the signal but provides no information as to when they appear. For music, for example, we know which notes are played but not in what order. We then introduce the wavelet transform; instead of using infinite sinusoidal waves, we use a function called the mother wavelet, localized in time, which starts and ends at zero. We slide this function along the signal when events occur. We compress or dilate this function: if we compress the function, it becomes very short

[4]It is important to note that the calculations were performed by a Python notebook on a single core of a 12th generation i5 CPU. Performance could therefore be significantly improved by a change of hardware.

Input Representation Type	Execution Time (Random Forest - Classic)
Raw Signals (*Raw*)	365.09 s ($\sim$ 6 min 05)
Activation time	717.23 s ($\sim$ 11 min 57)
Peaks	85.78 s ($\sim$ 1 min 25)
Features	5.61 s
Reconstruct	334.14 s ($\sim$ 5 min 34)

Table 10: Computation time of the Random Forest according to the form of the input signal (Classic Sort).

and allows for the analysis of high frequencies. If we dilate it, it lengthens and allows for the analysis of the low frequencies of the signal, i.e., global trends.

In practice, we adapt the Matlab code from [6] to our problem in Python and draw inspiration from [9]. To characterize ECG signals for classification, we adopt an approach combining temporal, frequency, and multifractal descriptors. The extraction is carried out at two distinct time scales:

- Local analysis (by blocks): Each signal is segmented into 8 blocks of approximately one minute. On each block, three types of characteristics are calculated: the coefficients of an autoregressive (AR) model of order 4 to capture temporal dynamics, and the Shannon entropy derived from a discrete wavelet packet transform to measure the local regularity of the signal.
- Global analysis: Over the entire signal, the variance of multi-scale wavelets is estimated in an unbiased manner. Using the 'db2' wavelet and excluding coefficients affected by boundary conditions, this variance is calculated over a total of 14 resolution levels.

This combination of indicators provides a robust and recognized mathematical signature for discriminating cardiac pathologies. For greater clarity, we henceforth refer to all of these indicators as special indicators.

An autoregressive model is an algorithm that attempts to guess the value of the signal at a time t based on previous values. It is said to be of order 4 if it uses the 4 previous values to determine the value of the signal at the current instant, which can be formalized as: $x_t \approx a_1 x_{t-1} + a_2 x_{t-2} + a_3 x_{t-3} + a_4 x_{t-4}$ where the a_i are the model coefficients, which is what we keep. They can detect rhythm changes, for example. We calculate Shannon entropy on the wavelet coefficients. Low entropy indicates that energy is concentrated on a few specific wavelet coefficients, indicating that the heart beats regularly. Conversely, high entropy shows a dispersion of energy and a more chaotic signal. The so-called 'db2' wavelet is the second wavelet of a very famous family: the Daubechies family. It is composed of asymmetrical peaks that make it particularly suitable for the study of ECG signals (see the representation of Daubechies wavelets in the appendix).
We first calculate these indicators on the signals. Then, we apply the Random Forest classifier, obtaining an error rate of approximately 0.24, as well as Table 11. We obtain rather poor results proceeding this way, which can be attributed, at least in part, to the conversion of the Matlab code to Python. We decide to test this procedure on activation times; we obtain an error rate of only 0.06 and display the score table, see 12. All scores are high; we can clearly distinguish the 3 types of classes without having to pair them. We even obtain better results than the source [6], which had an error rate of 0.8 and lower scores.

Class	Precision	Recall	F1-Score
A	73.554	92.708	82.028
C	100.000	20.000	33.333
N	80.000	77.778	78.873

Table 11: Evaluation of classification performance by signal class for raw signals and special indicators.

Class	Precision	Recall	F1-Score
A	96.806	94.792	95.789
C	89.655	86.667	88.136
N	89.744	97.222	93.333

Table 12: Evaluation of classification performance by signal class for activation times and special indicators.

3.7 Summary of Random Forest Performance

Having previously established the superiority of Random Forest over other classifiers, we provide a synthesis of these performances in Table 13. Based on this table, we show that the inputs yielding the best results are, in this order, features and activation times. The exception is the nsr1 sort, for which activation times give the best result. The error ratio of 7.8% is very low (one of the only ones to drop below the 10% mark). However, this result must be slightly qualified, as the recall score for CHF signals is only 57%. This value may be due to a flaw in our data, namely class imbalance; as a reminder, we have 96 ARR-type signals, 36 NSR-type signals, and only 30 CHF-type signals. This disparity increases even further in the context of the nsr1 sort, where ARR and NSR signals are merged under the NSR label. This implies a low representation of CHF signals in the test set, meaning one or two misclassifications will impact the CHF or NSR scores much more heavily. Another remarkable result is that given by features with the chf sort, since we obtain a rate of 4.45%, which is the lowest of all, with high scores across the board. Furthermore, this also indicates that information seems concentrated at the level of activation times for CHF signals but not for the other types. Finally, the special indicators z, which correspond to special indicators calculated on activation times, give an excellent error rate of 6%. A fortiori, since this rate corresponds to a classic sort and is accompanied by very good scores (the results being satisfactory in themselves, we did not apply this method to the other sorts). This result is all the more remarkable as it is consistent with the results of the other inputs. Indeed, this method combines the use of activation times and feature extraction; even though these are not the same characteristics as for the features, they fulfill the same role of providing both specific and general information. We can see through the F1-scores that activation times are generally more effective for determining CHF-type signals, while features perform better on ARR and NSR signals.

Input	Class	Error Rate (%)	Precision (%)	Recall (%)	F1-Score (%)
— SORTING METHOD: CLASSIC —					
Raw	A	28.09	68.07	98.28	80.35
	C		14.00	2.37	4.05
	N		95.50	60.93	72.85
Activation time	A	17.33	78.85	97.06	86.85
	C		98.92	58.97	71.96
	N		91.38	65.29	74.84
Peak	A	37.91	69.76	70.64	69.86
	C		33.84	21.06	24.98

Input	Class	Error Rate (%)	Precision (%)	Recall (%)	F1-Score (%)
	N		55.96	72.91	62.66
Features	A	15.91	82.97	96.20	87.17
	C		76.43	54.41	61.01
	N		96.08	87.21	90.72
Reconstruct	A	28.09	68.06	98.64	80.46
	C		20.50	4.01	6.56
	N		96.25	58.30	70.30
Special Indicators z	A	6.00	83.67	93.79	88.44
	C		95.49	75.15	84.11
	N		88.14	78.40	83.00
— SORTING METHOD: CHF —					
Activation time	C	12.76	85.95	99.84	92.33
	N		99.22	46.32	61.37
Peak	C	19.45	90.97	83.03	86.58
	N		58.19	72.41	63.54
Features	C	4.45	95.64	98.81	97.16
	N		96.04	85.00	89.60
Reconstruct	C	15.67	83.70	99.04	90.66
	N		91.65	36.93	50.32
— SORTING METHOD: NSR1 —					
Activation time	C	7.88	99.75	57.01	71.03
	N		91.34	99.96	95.42
Peak	C	21.67	31.61	17.18	20.93
	N		83.33	91.96	87.34
Features	C	11.79	74.19	54.90	60.80
	N		90.75	95.56	93.01
Reconstruct	C	16.73	45.50	9.24	15.07
	N		83.26	99.63	90.70
— SORTING METHOD: NSR2 —					
Activation time	A	24.45	76.65	84,65	80.07
	N		75.73	63.01	67.75
Peak	A	33.61	71.64	70.86	70.88
	N		60.24	60.12	59.56
Features	A	14.39	85.64	91.24	88.06
	N		87.51	77.79	81.60
Reconstruct	A	28.00	72.76	83.36	77.50
	N		72.16	56.16	62.41

Table 13: Summary of Random Forest performance.

Conclusion

This study compared different approaches for information extraction and classification of electrocardiogram signals. The main objective was to study the value of Convolutional Dictionary Learning (CDL) applied to this problem. CDL applied to a signal decomposes it into a small number of elementary building blocks, called atoms, and their corresponding activation times. Here, we use a CDL that detects three atoms per signal type. We compared this method against other signal representations: using raw data, signal peaks, a set of basic signal features, and special indicators based on the wavelet transform and autoregression.

We applied these methods to an ECG signal dataset divided into three categories: ARR (ar-

rhythmia), CHF (heart failure), and NSR (healthy). To classify the signals, we applied a set of classifiers to their different representations. We compared signal representations and classifiers by analyzing prediction error rates, precision, recall, and F1-scores calculated for each signal type. By comparing the obtained results, we were able to make the following observations: attempting to classify signals into three distinct groups from the outset is highly unproductive and leads to large errors. It is better to adopt an approach where we cross-reference classifications using a one-versus-all system, in which we combine two types of signals and oppose them to the third. The classifier yielding the best results is scikit-learn's RandomForest. As expected, passing raw signals as inputs to the classifiers yields very poor results, and the same goes for peaks. Basic features, on the other hand, provide the best results most of the time. For instance, we achieve a remarkable result (below a 10% error rate) when opposing ARR and CHF signals to NSR signals. Activation times associated with CDL sometimes yield better results, notably in the configuration where we oppose ARR and NSR signals to CHF signals (dropping below a 10% error rate). We can then assume that for CHF signals, the information is contained within the activation times, whereas for ARR and NSR signals, it is found more in the features. The complementarity of these signal representations led us to calculate special indicators directly on activation times instead of doing so on raw signals (which gives poor results). We applied the RandomForest classifier to them and obtained the best results of the study, combining a very low error rate (well below the 10% mark) for a direct classification across the three signal types, along with high scores for each type.

These results pave the way for more reliable automated diagnostics, although larger-scale testing—as we probably had too few CHF signals, for example (only 30)—as well as the thorough exploration of conditioning methods like wavelets, remain promising perspectives to enrich this protocol.

References

[1] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pages 108–122, 2013.

[2] Mainak Jas, Thomas Dupré La Tour, Umut Şimşekli, and Alexandre Gramfort. Learning the morphology of brain signals using alpha-stable convolutional sparse coding. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1099–1108, 2017.

[3] Lakshmi Kuruguntla, Mamatha Bandaru, Dokku Tejaswi, Anup Kumar Mandpura, Sravan Kumar Sikhakoli, and Vineela Chandra Dodda. Geophysical data denoising using dictionary learning method with ramanujan sums for oil and minerals exploration. *Artificial Intelligence in Geosciences*, 6(2):100168, 2025.

[4] Jérôme MOLINARO. Analyse rapide et simplifiée de l'électrocardiogramme, 2022.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[6] The MathWorks, Inc. Ecg classification using wavelet features, 2026.

[7] The MathWorks, Inc. Introduction aux familles d'ondelettes, 2026.

[8] Tom Dupré La Tour, Thomas Moreau, Mainak Jas, and Alexandre Gramfort. Multivariate convolutional sparse coding for electromagnetic brain signals, 2018.

[9] Qibin Zhao and Liqing Zhang. Ecg feature extraction and classification using wavelet transform and support vector machines, 11 2005.

Appendix

Here we provide the complete Python code developed for this study, which can be downloaded and used at the reader's convenience, along with the ECG data used.

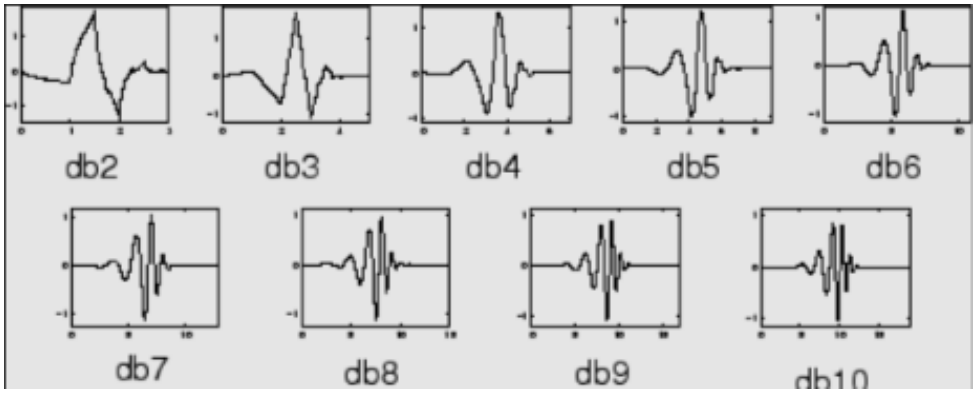


Figure 9: Daubechies wavelets[7]